

[LOG]

[IDS]

[SIEM]

[XSS]

[SCAN]

Building a SOC Detection Lab

with Wazuh SIEM + Suricata IDS on Rocky Linux

- Suricata 8.0 IDS deployment with Emerging Threats ruleset
- Rocky Linux 9.3 network sensor + Wazuh Agent integration
- Real-time Nmap port scan detection (195 alerts)
- DVWA web application setup for attack simulation
- XSS Cross-Site Scripting detection (197 alerts)
- End-to-end SOC pipeline with eve.json log forwarding

Prepared By:	Ayesha Information Security Analyst
Organization:	Vital Medicure Labs
Platform:	Rocky Linux 9.3 + Kali Linux + VMware
Date:	May 6, 2026

Table of Contents

01 Suricata 8.0 Installation on Rocky Linux

dnf copr / OISF repository / RPM install

02 Emerging Threats Rules Configuration

rules download / tar extraction / file permissions

03 Suricata YAML Configuration

HOME_NET / af-packet / rule-files / interface

04 Suricata Service — Active & Running

systemctl status / process verification

05 Wazuh Agent Installation

RPM install / WAZUH_MANAGER enrollment

06 Suricata Log Forwarding to Wazuh

ossec.conf localfile / eve.json / JSON format

07 Agent Verified in Wazuh Dashboard

RockyLinux-Suricata2-9.3 active / ID 002

08 Nmap Port Scan Attack from Kali

nmap -sS -Pn / stealth scan / target enumeration

09 Wazuh Alerts — Port Scan Detection

195 hits / ET SCAN rules / SQL MySQL VNC

10 DVWA Installation on Kali Linux

lamCarron script / Apache / MariaDB / PHP

11 DVWA Login & Dashboard Access

150.1.7.102/DVWA / admin:password / security low

12 XSS Attack & Suricata Detection

script tag injection / ET WEB_SERVER rule / 197 hits

13 Lab Summary & Results

full pipeline verified / SOC detection confirmed

[\[DNF\]](#) [\[COPR\]](#) [\[OISF\]](#) [\[RPM\]](#)

Suricata 8.0 Installation on Rocky Linux

dnf copr

epel-release

suricata

STEP 01

Installing Suricata 8.0 via OISF COPR Repository

Suricata 8.0 was installed on Rocky Linux 9.3 using the official OISF COPR repository. This is the recommended installation method for enterprise Linux distributions as it provides the latest stable build of Suricata directly from the Open Information Security Foundation. EPEL (Extra Packages for Enterprise Linux) was also enabled as a dependency source.

Step 1 — Enable COPR Plugin & Repository:

```
sudo dnf install 'dnf-command(copr)' -y  
  
sudo dnf copr enable @oisf/suricata-8.0 -y
```

Step 2 — Install EPEL Release & Suricata:

```
sudo dnf install epel-release -y  
  
sudo dnf install suricata -y
```

Key Observations

- dnf-command(copr) plugin allows enabling community repositories
- @oisf/suricata-8.0 is the official OISF maintained COPR repo
- EPEL provides additional dependency packages for Rocky Linux
- Suricata 8.0.4 installed — latest stable release from OISF
- sudo privileges required — aisha added to wheel group for access

[RULES] [ET OPEN] [YARA] [CURL]

Emerging Threats Rules Configuration

mkdir	curl	tar	chown	chmod
-------	------	-----	-------	-------

STEP 02

Deploying Emerging Threats Open Ruleset

The Emerging Threats (ET) Open ruleset is a community-maintained collection of Suricata/Snort compatible detection rules. It covers thousands of threat categories including port scans, malware C2, web attacks, SQL injection, XSS, and more. The rules were downloaded and deployed to the Suricata rules directory with proper file ownership and permissions.

Step 3 — Create Rules Directory & Download ET Rules:

```
sudo mkdir -p /etc/suricata/rules/

cd /tmp/

sudo curl -LO
https://rules.emergingthreats.net/open/suricata-8.0.4/emerging.rules.tar.gz

sudo tar -xvzf emerging.rules.tar.gz

sudo mv rules/*.rules /etc/suricata/rules/

sudo chown -R suricata:suricata /etc/suricata/rules/

sudo find /etc/suricata/rules/ -name '*.rules' -exec chmod 640 {} \;
```

Rules Deployed (Sample):

Rule File	Category
emerging-scan.rules	Port scans & reconnaissance
emerging-web_server.rules	Web server attacks, XSS, injections
emerging-sql.rules	SQL injection patterns
emerging-malware.rules	Known malware signatures
emerging-dos.rules	DoS & DDoS attack patterns
emerging-exploit.rules	Exploit kit traffic

Key Observations

- 5,323 KB ruleset downloaded from Emerging Threats servers
- 50+ .rules files extracted covering all major attack categories
- chown -R suricata:suricata ensures Suricata process can read rules

- chmod 640 restricts rule files — readable by suricata group only
- find command used because shell wildcard expansion failed without sudo context

[YAML] [HOME_NET] [AF-PACKET] [RULES]

Suricata YAML Configuration

suricata.yaml	af-packet	HOME_NET	interface
---------------	-----------	----------	-----------

STEP 03

Configuring suricata.yaml & sysconfig

The main Suricata configuration file `suricata.yaml` was edited to define the protected network range, external network scope, rules path, and the network interface for packet capture. AF-Packet mode was selected for high-performance Linux packet capture. The `sysconfig` file was also configured to bind Suricata to the correct network interface on startup.

suricata.yaml — Key Configuration:

```
# Network scope

HOME_NET: "[150.1.7.0/24]"

EXTERNAL_NET: "any"


# Rules

default-rule-path: /etc/suricata/rules

rule-files:

- "*.rules"


# Capture interface (Linux high-speed mode)

af-packet:

- interface: ens192
```

/etc/sysconfig/suricata — Startup Options:

```
OPTIONS="-i ens192 --user suricata"
```

Key Observations

- HOME_NET set to 150.1.7.0/24 — the entire lab subnet is protected
- EXTERNAL_NET: any — alerts fire on traffic from any external source
- AF-Packet mode provides kernel-bypass packet capture for performance
- ens192 is the lab network interface (VMnet1 connected NIC)
- --user suricata drops privileges after startup for security

[SYSTEMCTL] [ACTIVE] [PID] [RUNNING]

Suricata Service — Active & Running

systemctl enable

systemctl restart

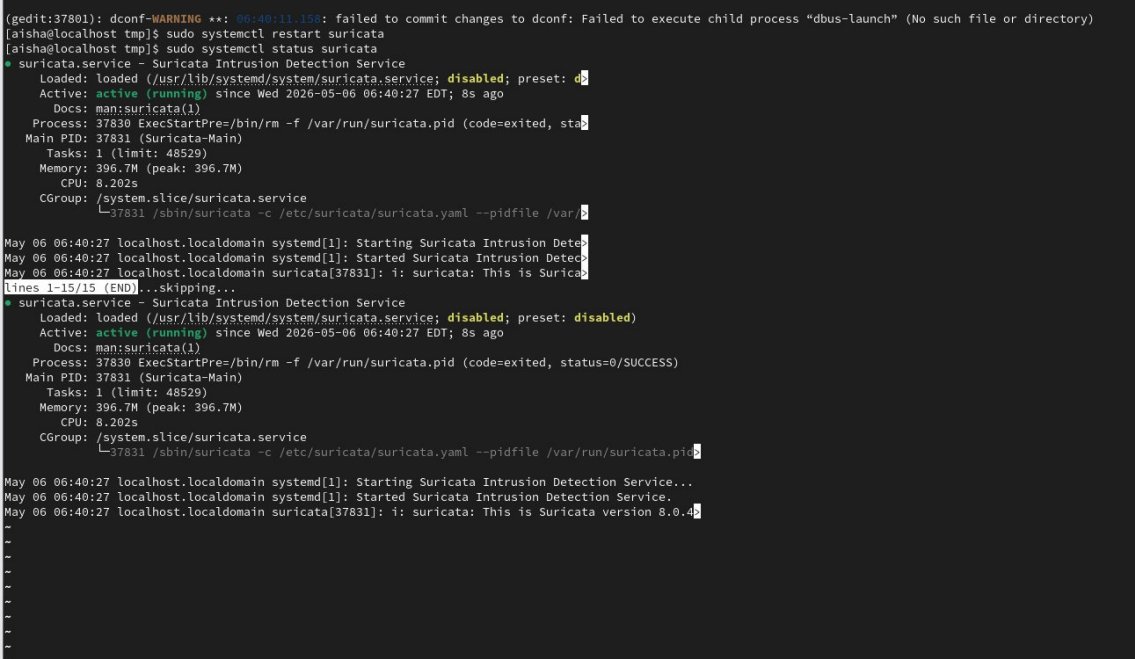
systemctl status

STEP 04

Starting & Verifying Suricata Service

After completing the configuration, Suricata was enabled as a systemd service and started. The service status was verified to confirm it was active and running. Suricata 8.0.4 successfully loaded all Emerging Threats rules and began monitoring the ens192 interface.

```
sudo systemctl enable suricata
sudo systemctl restart suricata
sudo systemctl status suricata
```



Screenshot 04 — Suricata 8.0.4 active (running) on Rocky Linux 9.3 | PID 37831 | Memory 396.7M

Key Observations

- Suricata version 8.0.4 confirmed in service log output
- Active (running) status confirmed — service started successfully
- Main PID 37831 — Suricata-Main process running
- Memory usage: 396.7M — all ET rules loaded into memory
- CPU: 8.202s startup time — indicates full rule compilation

[\[WAZUH\]](#) [\[RPM\]](#) [\[AGENT\]](#) [\[ENROLLMENT\]](#)

Wazuh Agent Installation

curl

rpm -ihv

systemctl

WAZUH_MANAGER

STEP 05

Installing Wazuh Agent & Registering to Manager

The Wazuh Agent v4.14.4 was installed on the Rocky Linux Suricata machine. During installation, the WAZUH_MANAGER environment variable was used to automatically register the agent with the Wazuh Manager at 150.1.7.99. The agent was named 'RockyLinux-Suricata2-9.3' for identification in the Wazuh dashboard.

```
# Download and install Wazuh Agent

curl -o wazuh-agent-4.14.4-1.x86_64.rpm \

https://packages.wazuh.com/4.x/yum/wazuh-agent-4.14.4-1.x86_64.rpm


# Install with auto-enrollment to manager

sudo WAZUH_MANAGER='150.1.7.99' \

WAZUH_AGENT_NAME='RockyLinux-Suricata2-9.3' \

rpm -ihv wazuh-agent-4.14.4-1.x86_64.rpm


# Enable and start agent

sudo systemctl daemon-reload

sudo systemctl enable wazuh-agent

sudo systemctl start wazuh-agent
```


[OSSEC.CONF] [EVE.JSON] [LOCALFILE] [JSON]

Suricata Log Forwarding to Wazuh

ossec.conf	localfile	log_format json	eve.json
------------	-----------	-----------------	----------

STEP 06

Configuring ossec.conf for Suricata eve.json

The Wazuh Agent configuration file `ossec.conf` was edited to add a `localfile` block that instructs the agent to monitor Suricata's `eve.json` log file. EVE (Extensible Event Format) is Suricata's unified JSON output format containing all alerts, flows, HTTP, DNS, and file events. Setting `log_format` to `json` enables Wazuh to parse and index each event correctly.

```
sudo gedit /var/ossec/etc/ossec.conf

# Add inside <ossec_config> block:

<localfile>

<log_format>json</log_format>

<location>/var/log/suricata/eve.json</location>

</localfile>

# Restart agent to apply changes

sudo systemctl restart wazuh-agent
```

Key Observations

- `eve.json` is Suricata's primary alert output — contains all detection events
- `log_format: json` tells Wazuh to parse each line as a JSON object
- Wazuh agent monitors the file continuously and forwards new events
- Events are correlated with Wazuh rules (rule ID 86600 for Suricata)
- All alerts visible in Wazuh Discover under agent `RockyLinux-Suricata2-9.3`

[DASHBOARD] [ACTIVE] [AGENT-002] [ENDPOINT]

Agent Verified in Wazuh Dashboard

Agents Management

Summary

Deploy new agent

Active

STEP 07

RockyLinux-Suricata2-9.3 Confirmed Active

After starting the Wazuh agent, the Wazuh dashboard was accessed to confirm successful agent registration. The agent RockyLinux-Suricata2-9.3 appeared as Agent ID 002 with Active status, running Rocky Linux 9.7, connected to cluster node node01 at 150.1.7.140.

Agent Name	RockyLinux-Suricata2-9.3
Agent ID	002
IP Address	150.1.7.140
Operating System	Rocky Linux 9.7
Wazuh Version	v4.14.4
Cluster Node	node01
Status	Active

Key Observations

- Agent registered automatically using WAZUH_MANAGER env variable during RPM install
- Agent ID 002 assigned by Wazuh Manager upon successful enrollment
- Active status confirms bidirectional communication with manager at 150.1.7.99
- Win10 Agent (ID 001) also visible but shows Disconnected status
- Both agents visible under Agents Management > Summary in Wazuh dashboard

[NMAP] [STEALTH SCAN] [KALI] [-sS -Pn]

Nmap Port Scan Attack from Kali

nmap -sS	-Pn	150.1.7.103	150.1.7.140
----------	-----	-------------	-------------

STEP 08

Launching Stealth Scan to Trigger Suricata Alerts

Nmap stealth SYN scans (-sS) were launched from Kali Linux (150.1.7.102) against two targets in the lab network. The -Pn flag skips host discovery, treating all hosts as online. This type of scan sends SYN packets to common ports and records which ports respond — a technique used in real-world reconnaissance to map services on target systems.

Attack Commands (executed on Kali Linux):

```
# Scan target 1

nmap -sS -Pn 150.1.7.103

# Scan target 2 (Suricata machine itself)

nmap -sS -Pn 150.1.7.140
```

```
└─# nmap -sS -Pn 150.1.7.103
nmap -sS -Pn 150.1.7.140
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-06 07:01 -0400
Nmap done: 1 IP address (0 hosts up) scanned in 1.48 seconds
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-06 07:01 -0400
Nmap scan report for 150.1.7.140
Host is up (0.00055s latency).
Not shown: 988 filtered tcp ports (no-response), 10 filtered tcp ports (admin-prohibited)
PORT      STATE SERVICE
22/tcp    open  ssh
9090/tcp   closed zeus-admin
MAC Address: 00:0C:29:73:9F:19 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 5.70 seconds

(root@kali)-[/home/kali]
└─# sudo bash -c "$(curl --fail --show-error --silent --location https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"
```

Screenshot 08 — Nmap stealth scan from Kali Linux | Scan result for 150.1.7.140 showing SSH port 22 open

Key Observations

- -sS (SYN scan) sends SYN packets — stealthier than full TCP connect scan
- -Pn skips ping — treats host as online even if ICMP is blocked
- 150.1.7.140: Port 22 (SSH) open, Port 9090 (zeus-admin) closed
- Nmap completed in 5.70 seconds — 988 filtered ports detected

- Scan immediately triggered Suricata ET SCAN rules in Wazuh

[ET SCAN] [195 HITS] [WAZUH] [SURICATA]

Wazuh Alerts — Port Scan Detection

ET SCAN

Oracle 1521

MSSQL 1433

VNC 5800

MySQL 3306

STEP 09

195 Real-Time Alerts Fired in Wazuh Dashboard

The Nmap stealth scan triggered 195 Suricata alerts which were immediately forwarded to the Wazuh dashboard via the agent's eve.json log forwarding pipeline. Each alert corresponds to a specific Emerging Threats SCAN rule matching suspicious inbound connections to well-known database and remote access ports — classic indicators of reconnaissance activity.



Screenshot 09 — Wazuh Discover showing 195 Suricata alerts from Nmap port scan | agent 150.1.7.140

Alerts Triggered:

Suricata Alert	Port	Category
ET SCAN Suspicious inbound to Oracle SQL	1521	Database Recon
ET SCAN Suspicious inbound to MSSQL	1433	Database Recon
ET SCAN Potential VNC Scan	5800-5820	Remote Access Recon
ET SCAN Suspicious inbound to PostgreSQL	5432	Database Recon
ET SCAN Suspicious inbound to mySQL	3306	Database Recon

[DVWA] [APACHE] [MARIADB] [PHP]

DVWA Installation on Kali Linux

curl install script

Apache2

MariaDB

PHP 8.4

STEP 10

Deploying DVWA for Web Application Attack Simulation

DVWA (Damn Vulnerable Web Application) is a deliberately insecure PHP/MySQL web application designed for security professionals to practice web application attacks in a legal environment. It was installed on Kali Linux using the IamCarron automated installation script which handles Apache2, MariaDB, PHP, and all dependencies automatically.

```
# Run automated DVWA installer on Kali Linux

sudo bash -c "$(curl --fail --show-error --silent --location \
https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"
```



```
(root@kali) ~/home/kali
# sudo bash -c "$(curl --fail --show-error --silent --location https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"

DVWA
INSTALLER

Welcome to the DVWA setup!
Script Name: Install-DVWA.sh
Author: IamCarron
Github Repo: https://github.com/IamCarron/DVWA-Script
Installer Version: 2.0

Updating repositories...
Get:1 https://download.docker.com/linux/debian bullseye InRelease [43.0 kB]
Get:3 https://pkg.claude-desktop-debian.dev stable InRelease
Get:2 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 https://pkg.claude-desktop-debian.dev stable/main arm64 Packages [427 B]
Get:5 http://kali.download/kali kali-rolling/main amd64 Packages [21.1 MB]
Get:6 https://pkg.claude-desktop-debian.dev stable/main amd64 Packages [425 B]
Get:7 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.4 MB]
Get:8 http://kali.download/kali kali-rolling/contrib amd64 Packages [118 kB]
Get:9 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [262 kB]
Get:10 http://kali.download/kali kali-rolling/non-free amd64 Packages [189 kB]
Fetched 75.2 MB in 27s (2,804 kB/s)
195 packages can be upgraded. Run 'apt list --upgradable' to see them.
Verifying and installing necessary dependencies...
apache2 is installed!
mariadb-server is installed!
mariadb-client is installed!
php is installed!
php-mysql is installed!
php-gd is not installed. Installing it now...
The following packages were automatically installed and are no longer required:
```

Screenshot 10 — DVWA Installer running on Kali Linux | Updating repos and installing dependencies

```
Default credentials:
Username: root

Password: [No password just hit Enter]
Enter SQL user:
Enter SQL password (press Enter for no password):
SQL commands executed successfully.
Configuring DVWA ...
Configuring permissions ...
Configuring PHP ...
Enabling Apache ...
Restarting Apache ...
DVWA has been installed successfully. Access http://localhost/DVWA to get started.
Credentials:
Username: admin
Password: password

With ♥ by IamCarron

(root@kali)-[/home/kali]
# ss
```

Screenshot 10b — DVWA installation completed | Credentials: admin / password | Access at localhost/DVWA

Key Observations

- Apache2, MariaDB, PHP 8.4, php-mysql, php-gd all installed automatically
- DVWA cloned from GitHub to /var/www/html/DVWA/
- MariaDB configured with empty root password for lab environment
- Kali IP on lab network: 150.1.7.102 (eth1 interface)

[HTTP] [150.1.7.102] [ADMIN] [SECURITY LOW]

DVWA Login & Dashboard Access

150.1.7.102/DVWA

admin:password

Setup DB

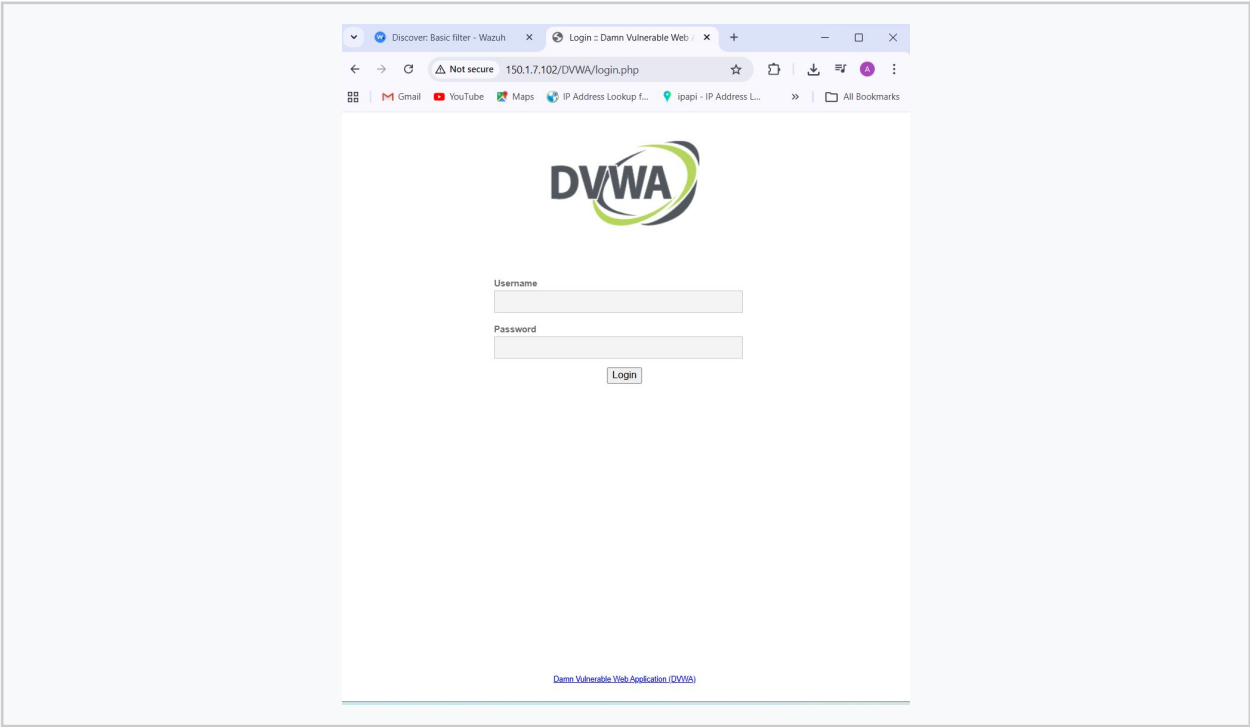
Security: Low

STEP 11

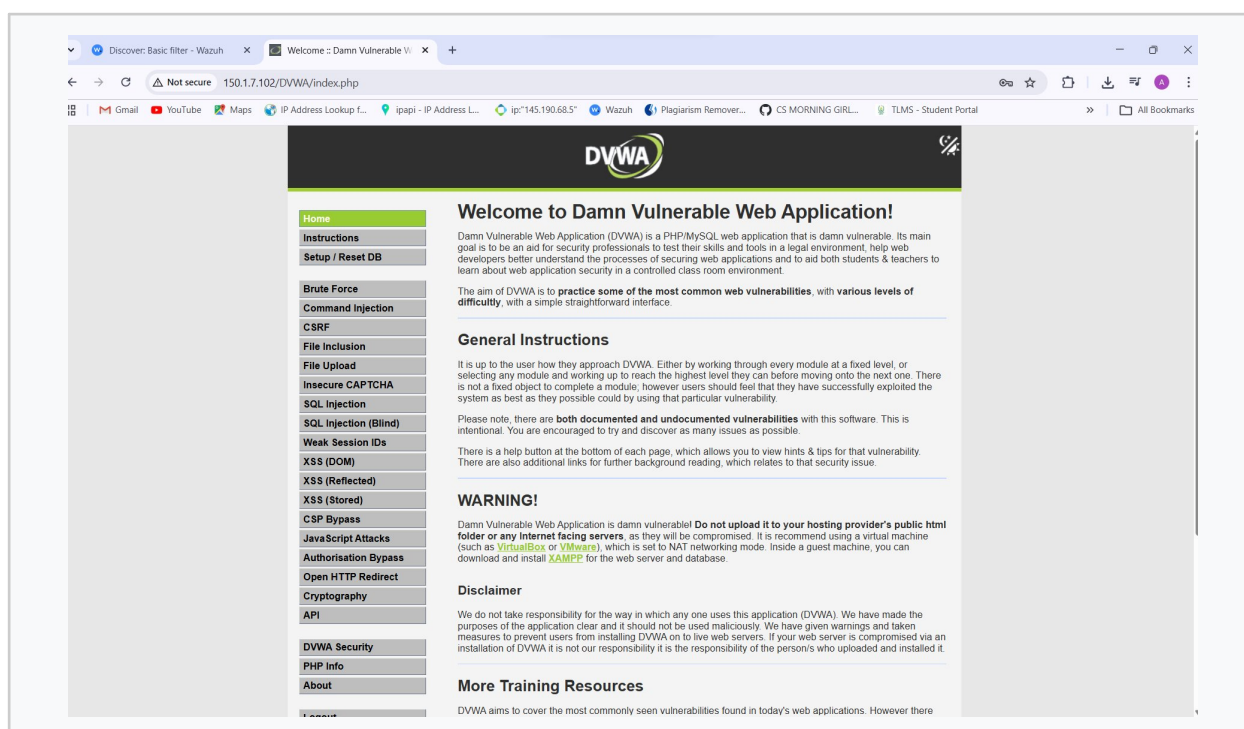
Accessing DVWA from Base Machine Browser

DVWA was accessed from the base machine browser using the Kali Linux lab IP address. After logging in with default credentials, the database was initialized via Setup/Reset DB, and the security level was set to Low to disable input filters — making all vulnerability modules exploitable for attack simulation and detection testing.

URL	http://150.1.7.102/DVWA
Username	admin
Password	password
Security Level	Low (all filters disabled)
Modules Available	SQL Injection, XSS, Command Injection, File Upload, Brute Force, CSRF...



Screenshot 11a — DVWA Login Page at 150.1.7.102/DVWA/login.php



Screenshot 11b — DVWA Dashboard showing all vulnerability modules | Security level set to Low

[XSS] [ET WEB_SERVER] [197 HITS] [SCRIPT TAG]

XSS Attack & Suricata Detection

XSS Reflected

script tag

ET WEB_SERVER

197 alerts

STEP 12

Cross-Site Scripting Attack Detected by Suricata

Cross-Site Scripting (XSS) attacks were executed against the DVWA XSS (Reflected) module. Script injection payloads were submitted in the input field, causing the server to reflect the script tag back in the HTTP response. Suricata's ET WEB_SERVER ruleset detected the script tag pattern in the URI and fired alerts which were immediately forwarded to Wazuh.

XSS Attack Payloads Used:

```
# Payload 1 – Classic script injection
```

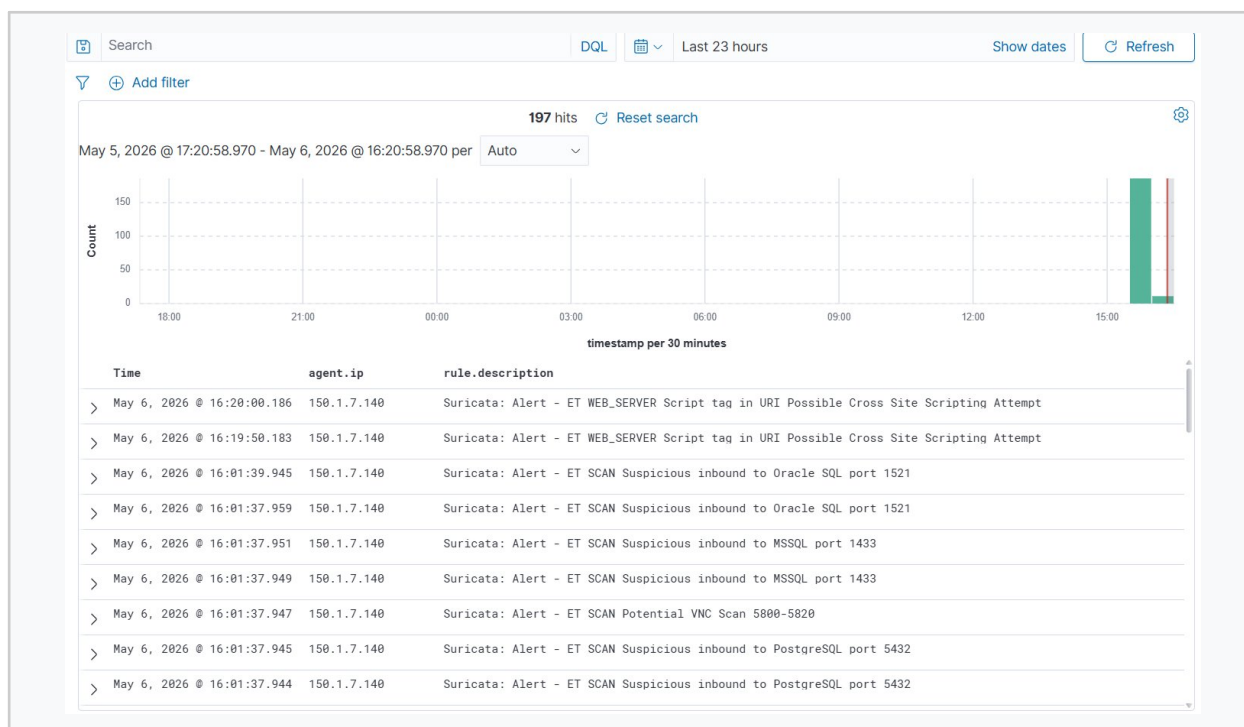
```
<script>alert('hello')</script>
```

```
# Payload 2 – Image tag onerror bypass
```

```
<img src=x onerror=alert('hello')>
```

```
# SQL Injection payload (also tested)
```

```
' OR '1'='1
```



Screenshot 12 — Wazuh Dashboard showing 197 hits including XSS detection alerts from Suricata

XSS Alerts in Wazuh:

```
Suricata: Alert - ET WEB_SERVER Script tag in URI XSS Attempt
```

```
Time: May 6, 2026 @ 16:20:00.186 | Agent: 150.1.7.140
```

```
Suricata: Alert - ET WEB_SERVER Script tag in URI XSS Attempt
```

```
Time: May 6, 2026 @ 16:19:50.183 | Agent: 150.1.7.140
```

Key Observations

- Suricata matched script tag pattern in HTTP URI against ET WEB_SERVER rules
- Two XSS alerts fired corresponding to the two payloads submitted
- Total alert count rose from 195 (port scan) to 197 (+ XSS) hits
- Modern browsers block XSS popups but Suricata still detects the network pattern
- Alert visible in Wazuh Discover under rule.description containing WEB_SERVER

[COMPLETE] [SOC] [IDS+SIEM] [VERIFIED]

Lab Summary & Results

Suricata 8.0

Wazuh v4.14

197 Alerts

100% Detected

STEP 13

End-to-End SOC Detection Pipeline Verified

This lab successfully demonstrated a complete SOC detection pipeline integrating Suricata IDS with Wazuh SIEM. All attack scenarios were detected and alerted in real time, confirming that the network-layer sensor (Suricata) and SIEM (Wazuh) work together effectively to provide comprehensive threat visibility across the lab network.

Step	Task	Result
01-04	Suricata 8.0 installed & running	PASS
05-06	Wazuh Agent installed & forwarding logs	PASS
07	Agent active in Wazuh dashboard	PASS
08-09	Nmap scan detected — 195 alerts	PASS
10-11	DVWA installed & accessible	PASS
12	XSS attack detected — 197 total alerts	PASS

Key Takeaways

- Suricata 8.0 with Emerging Threats rules provides comprehensive network threat detection coverage
- Wazuh SIEM + Suricata IDS integration creates an effective open-source SOC detection stack
- eve.json forwarding via ossec.conf localfile is a reliable method for SIEM integration
- Real-time alerts confirmed for both network-layer (Nmap) and application-layer (XSS) attacks
- DVWA provides a safe, legal environment to generate realistic web attack traffic for detection testing
- This pipeline can be extended with n8n automation for AI-powered alert triage and response

Ayesha | Information Security Analyst | Vital Medicure Labs | May 2026